Draw it or Lose it

CS 230 Project Software Design Template

Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 6

Document Revision History

Version	Date	Author	Comments
1.0	04/19/2026	Jorge Chavarria	Completed Project Three software design document and recommendations section.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room wants to expand *Draw It or Lose It* from Android to multiple operating systems. This document explains the best software design choices for that expansion. Because the game must support many users and game sessions, the system needs to be reliable, secure, and scalable.
A web-based distributed application is the best choice because it allows users on different devices to access the game more easily. Linux is the best server platform because it is flexible, secure, and cost-effective.

Requirements

The software must support multiple games, teams, and players at the same time. It should allow users on different operating systems to connect and play without major platform issues.
The application must also be secure, reliable, and able to grow as more users join. Since the game will run in different environments, the design should focus on compatibility and easy maintenance.

Design Constraints

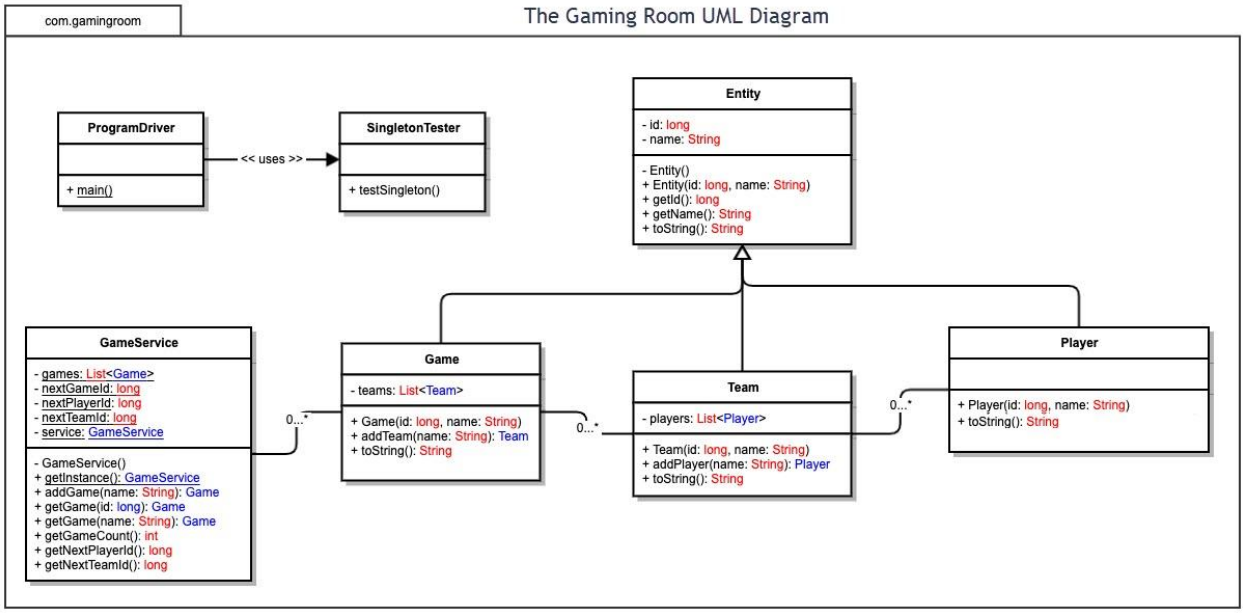
One constraint is that the game must work across multiple platforms, not just Android. This means the design cannot rely on one operating system only.
Other constraints include cost, security, performance, and network reliability. The system must be affordable, protect user information, and continue working well even when many users are connected.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The domain model includes the main classes: **Game, Team, and Player**. A game contains teams, and each team contains players.
This structure helps organize the software clearly and supports multiple game sessions at once. It also helps make sure names and data stay unique and organized.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can support server software, but it is not the best choice for this project. It is stable, but it usually costs more and is less common for large-scale server hosting.	Linux is the best server-side option because it is reliable, secure, scalable, and low cost. It works very well for web hosting and distributed systems.	Windows Server is a possible option, but it is usually more expensive because of licensing costs. It is capable, but Linux is a better fit for this project.	Mobile devices are not a good choice for hosting a web-based application because they have limited power, storage, and reliability. They are better used as client devices instead of server platforms for *Draw It or Lose It*.
Client Side	Mac users can access the game through a browser, which makes support easier. A web-based design keeps development simpler and avoids building a separate Mac app.	Linux clients can also use the game through a browser. This makes support easier, although testing across different Linux setups is still important.	Windows is one of the most common client platforms, so browser support is important. A web-based design works well and reduces development time.	Mobile support is important because many users play on phones and tablets. A mobile-friendly web app is a more efficient option than building separate native apps for each mobile platform.
Development Tools	Mac supports common development tools like Java, VS Code, Eclipse, and Git. These tools can be used to build and test the application.	Linux supports Java development, databases, Git, and server management tools very well. It is especially strong for hosting and deployment.	Windows also supports tools like Eclipse, VS Code, Git, and Java SDK. It works well for development, even if Linux is used for deployment.	For mobile devices, web technologies such as HTML, CSS, and JavaScript are the most practical tools. They make it easier to support many devices with one solution.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: Linux is the best operating platform for *Draw It or Lose It*. It is secure, scalable, reliable, and cost-effective. It also supports many types of clients connecting from different operating systems.

2. **Operating Systems Architectures**: Linux uses a kernel-based architecture that manages hardware, memory, processes, and networking. This is useful because the game must support many users and sessions at the same time.

3. **Storage Management**: A relational database such as MySQL or PostgreSQL is the best choice for storage. It can organize player accounts, teams, scores, and game sessions in an efficient way.

4. **Memory Management**: Linux uses virtual memory, paging, and process isolation. These techniques help the system stay stable and responsive while handling many users and active games.

5. **Distributed Systems and Networks**: The game should use a client-server distributed system. The server handles the main logic and storage, while clients connect through a network using TCP/IP and HTTP/HTTPS. This allows users on different platforms to access the same game.

6. **Security**: User information should be protected with HTTPS, TLS encryption, hashed passwords, and server access controls. Linux also provides security features like permissions, firewalls, and regular updates.